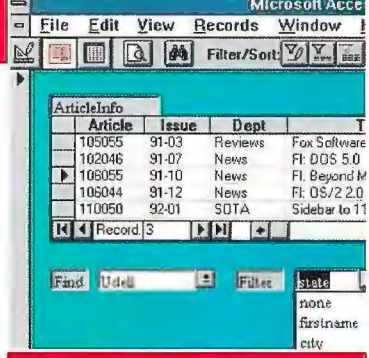
DECEMBER 1992 BYTE 66-MHz 486DX2s • CPU CHOICES • OBJECTS AND USERS Volume 17, Number 14

BYTE

THE WORLDWIDE COMPUTING AUTHORITY



FIRST LOOK: Microsoft Access

Winning Moves

How to Choose the Right CPU

REVIEWS:

Fastest PCs on the Planet: 66-MHz 486DX2s

HP LaserJet 4M: The 600-dpi Printer to Beat

STATE OF THE ART:

Objects Are Changing Computing, page 142

$3.50 U.S.A./$4.50 IN CANADA
A McGraw-Hill Publication/0360-5280

BYTE

December 1992 Volume 17, Number 14

COVER STORY

FEATURE

Make the Right CPU Move

PAGE 114

NEWS

30 MICROBYTES
SPARC-compatible workstations, priced similarly to high-end PCs, are expected by early 1993.

41 REPORT FROM ISRAEL
Coming In from the Cold
by Andy Redfern
As Mideast tensions wane, Israel can sell its wares more openly.

44 FIRST IMPRESSIONS
New Macs for the Desktop and Road
by Tom Thompson
Apple is updating computers and introducing new ones as rapidly as possible.

51 Microsoft's Windows Database
by Jon Udell
Microsoft Access, the happy union of SQL and Visual Basic.

52 Apricot XEN-LS II, is value-added worth it?

Video for Windows, coming soon to a PC near you

Convertible, pen computing without compromises

R4000 upgrade, a leap in power for SGI's Iris Indigo

Gateway 2000 4DX2-66V and **Hyundai 466D2,** low-cost DX2 local-bus systems

HP Vectra 486/33N, a low-cost system from an upscale vendor

72 WHAT'S NEW
Watch TV on the 486SX/25 Multimedia Computer System; centrally manage electrical power to local or remote LAN components with LanSafe II; and more.

FEATURES

114 Make the Right CPU Move
by Andy Redfern
New CPUs confuse PC buying decisions.
LAB TESTS: DOES BRAND MATTER?
by Raymond GA Côté
OTHER PLAYERS FIND NICHES
by Andy Redfern, Dave Andrews, Andy Reinhardt, and Tom Halfhill

129 TrueType A to Z
by Greg Loveria
Why TrueType is a key part of Windows 3.1.

137 Lower the Voltage, Raise the Power
by Jeffrey D. Shepard

STATE OF THE ART

OBJECTS FOR END USERS

142 Overview: Objects for End Users
by Cary Lu
Document-oriented computing is the logical culmination of the object-oriented revolution.
WHAT'S IN AN OBJECT?
by Mark Clarkson

153 Brave New Desktop
by Peter Wayner
Object technologies will let you shape your computing environment.
OBJECTS AND PENPOINT
by Mark Clarkson
OBJECT-ORIENTED CASE
by Daniel W. Rasmus

161 Relating to Objects
by Daniel W. Rasmus
Object technology finds a place in database management.

167 Objects of Substance
by Sergiu S. Simmel and Ivan Godard
Object storage is a natural for persistent-data servers.

172 Resource Guide: Object-Oriented Database Managers

COVER PHOTOGRAPHY: JIM SCHERER © 1992

REVIEWS

174 **SOLUTIONS FOCUS**
Grab Your Audience with Audio
by Tom Yager and Rick Grehan
With current sound boards, sequencers, and editors, you don't have to be a recording engineer to add audio to your presentations.

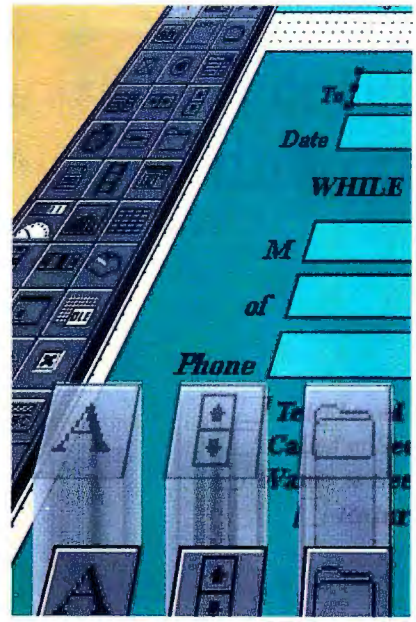
196 **Windows BASICs** ▷
by Mike Wiggins
Three Windows BASIC programs square off.

205 **Clocking the Fastest PCs on the Planet**
by Steve Apiki
Performance comparison of 66-MHz DX2 systems from Compaq, Dell, ALR, and NEC.

209 **A New LaserJet, a New Standard**
by Shelley Cryan
Hewlett-Packard's LaserJet 4M should shake up the printer market.

213 **TypeReader Takes OCR Toward Better Recognition**
by D. Barker
ExperVision's TypeReader OCR software is a step toward better recognition.

217 **The Phaser II SD Prints Dazzling Dyes**
by Tom Thompson
Tektronix's new dye-sublimation printer costs under $10,000.



224 **Stepping Up to XVT 3.0**
by Raymond GA Côté
The latest XVT marks interface library improvements and new development tools.

225 **The Windows File Shuffle**
by Stanford Diehl and Dana Hudes
File conversion under Windows solves the complex problem of divergent graphics formats.

232 **Reviewer's Notebook: A Real-World Notebook Battery Test**
by Howard Eglowstein
Notebook battery-life testing.

HANDS ON

237 **UNDER THE HOOD**
The PCMCIA Redefines Portability
by Roger C. Alford
The PCMCIA 2.0 PC card standard opens a new world for subnotebook and hand-held computers.

247 **SOME ASSEMBLY REQUIRED**
Writing a Windows DLL
by Randall A. Nagy
Here's a simple DLL-based debugging aid for Windows.

256 **SOFTWARE CORNER**
Audit Your LAN
by Barry Nance, Tom Thompson, and Ben Smith
Easy LAN inventory, a crash disk for System 7.0, and face-saving Unix utilities.

257 **BEYOND DOS**
Object-Oriented DLLs
by Gen Kiyooka
Build reusable objects with Windows DLLs.

260 **ASK BYTE**
The BYTE Lab responds on clock inconsistencies, loopback plug pin-outs, and other issues.

OPINIONS

97 **USER'S COLUMN**
A Computer in the Hand
by Jerry Pournelle
Palmtops, PCMCIA, virus newsletters, and the BBS scene.

234 **BOOK AND CD-ROM REVIEWS**
Photographic Lies
by Hugh Kenner, Tom Yager, Tom Thompson, Jon Udell, and Stanford Diehl
The search for visual truth, design tips, TCP/IP explainer, the times of physicist Richard Feynman, and more.

324 **STOP BIT**
Technology and the New World Order
by James Burke
Advancing technology is creating a new, more complex social order.

12 **EDITORIAL**
Testing and CPUs
by Dennis Allen

22 **LETTERS**
Unix lives!

READER SERVICE
322 Editorial Index by Company
318 Alphabetical Index to Advertisers
320 Index to Advertisers by Product Category
Inquiry Reply Cards: **208A, 320A**

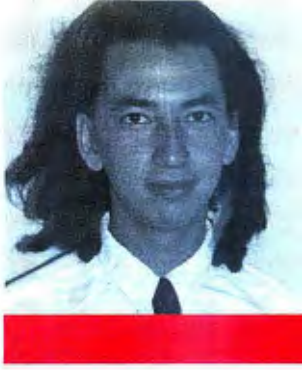
265 **BUYER'S GUIDE**
Mail Order
Hardware/Software Showcase
Buyer's Mart

PROGRAM LISTINGS
From BIX: Join "listings/frombyte92"

INSIDE BYTE

BYTE (ISSN 0360-5280) is published monthly with additional issues in October and November by McGraw-Hill, Inc. U.S. subscriber rate $29.95 per year. In Canada and Mexico, $34.95 per year. European surface mail subscriptions $50, airmail $70. Non-European subscriptions, $50 surface mail or $75 airmail. All foreign subscriptions are payable in U.S. funds that can be drawn on a U.S. bank. Single copies $3.50 in the U.S., $4.50 in Canada. Executive, Editorial, Circulation, and Advertising Offices: One Phoenix Mill Lane, Peterborough, NH 03458. Second-class postage paid at Peterborough, NH, and additional mailing offices. Postage paid at Winnipeg, Manitoba. Registration number 9321. Registered for GST as McGraw-Hill, Inc., GST #123075673. Printed in the United States of America. **Postmaster**: Send address changes and fulfillment questions to BYTE Subscriptions, P.O. Box 552, Hightstown, NJ 08520.

CHESS SET COURTESY OF THE LONDON HARNESS COMPANY, BOSTON, MASSACHUSETTS



GEN KIYOOKA

OBJECT-ORIENTED DLLS

Some say that Windows' architecture is less than elegant, but I still marvel at the transformation from Windows 3.0 to 3.1. A host of new functions and data types appeared: DDE management, OLE, common dialog boxes, and more. DLLs made the complexity manageable. Each new library added a group of APIs and, more important, a new object, class, or abstract data type. As a result, Windows, in the midst of growth, remained organized and comprehensible.

Of course, the design of operating-system software is an ambitious and difficult undertaking, so it makes sense that Windows' engineers would take the time to partition their problem space in a tasteful manner. But what of the ordinary Windows applications programmer? Is the extra effort really required?

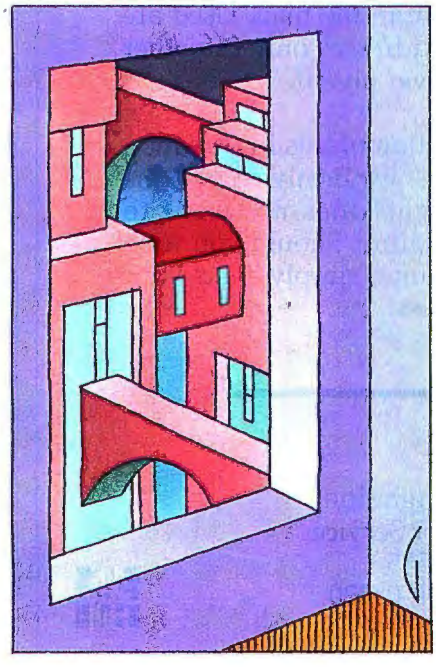
Consider Microsoft Word for Windows' import filters, which are organized as a set of DLLs. Don't be fooled by the .CNV filename extensions. If you run EXEHDR (the Microsoft C .EXE snooper) against Word's filters, you'll find that they're really DLLs that export functions. Even more interesting, you'll find that each filter exports a consistent set of functions: GETINIENTRY, INITCONVERTER, FOREIGNTORTF, RTFTOFOREIGN, and ISFORMATCORRECT.

When you ask Word to import a file whose format is foreign, it enumerates the filters listed in WIN.INI. When you pick one, Word loads the library and stuffs the addresses of its functions into a table of function pointers, neatly TYPEDEFed to enforce strict type checking. Knowing nothing of the filter's inner workings, Word simply asks it to perform a file conversion. After kicking things off with a call to INITCONVERTER, Word might then use ISFORMATCORRECT to verify the contents of the foreign file as an appropriate match for the DLL and FOREIGNTORTF to create an intermediate representation of the data in the RTF (Rich Text Format) that Word can use.

Once the requirements for a file converter are defined and documented, any Windows programmer can write an import filter. Because Windows itself defines parameter passing and calling conventions for DLLs, a filter can be written as easily in Pascal as in C. Interface elements such as dialog boxes, menus, and bit-map files can be bound into the module. Initialized, static data lives in its default data segment. If data packets are rigorously abstracted to HANDLE types, the private workings of the DLL and host application become opaque to one another.

From this perspective, a DLL is more than just a code repository. It's a distributable, reusable object. In the case of the WinWord import filter, that object abstracts the type *foreign file* represented by a file handle and wrapped with a template of method definitions that describe the operations on the type foreign file. The variation in type of the foreign file (i.e., its subclasses) is represented by instantiations of the DLL description—the individual .CNV files. Because the requirements of the interface between the converter and Word have been formalized, and the relationship between them conveniently decoupled via dynamic, run-time linking, any number of converters can be independently produced, tested, and distributed.

Dynamic link libraries encourage object-oriented design



To the delight of software engineers and academics alike, effective software design with DLLs shifts the emphasis toward early specification, abstraction, and interface design. Just any old DLL won't do. A correct interface is of paramount importance, because it governs how application components interact. A large software project may be designed and implemented as a collection of DLLs. With a rigorous specification, teams or individuals can work independently.

Designing DLLs

Designing a first DLL interface can be a daunting task. One approach is to study what other developers have done. All you need is the EXEHDR utility and a working version of the applications software. Even a crippled demonstration program retains the peculiar architecture of its commercial counterpart and, as such, makes for a low-cost study tool suitable for spelunking with EXEHDR. The Windows API, in all its splendor, is another suitable avenue for exploration.

For the ultimate in late binding, you can exploit run-time dynamic linking, as Word's import filters do. That means the calling application doesn't even discover the name of the DLL—let alone resolve its entry points—until run time.

Alternatively, you can bind the DLL's name into your program and resolve the

ILLUSTRATION: JOE GAST © 1992

DLL's entry points in advance, using the linker. In this model, you can alter your program's behavior by swapping in any DLL that has the right name and exports the right set of functions. The name of the module becomes an attribute of the formal definition of the interface. This technique is less flexible than the run-time approach, but it's also simpler. You don't have to build and maintain a list of function pointers, because the linker does that for you. Use this approach to distribute incremental software updates or to outfit an application for multiple versions. Consider, for example, a network-aware application that can run over NetBIOS, SPX/IPX, or TCP/IP protocols. You might choose among protocol modules during installation and never again need to change that DLL.

In larger projects, you'll have to make a basic design decision. Should your DLL export a few complex functions or many simple ones? I've just lent a helping hand to an application design that required such a decision. I helped the architect weigh the pros and cons of each approach. In this case, it was necessary to design a DLL that would expose nearly every aspect of the application's behavior to modification by the user. We decided that a single polymorphic entry point would serve our needs best.

This entry point would take just two parameters: an integer representing an event and a 32-bit word whose meaning depended on the event passed in the first parameter. At every stage in the execution of the application, we would arrange to send events to the DLL by way of this polymorphic entry point. The definition of each event would include a definition of the data passing through in the 32-bit parameter.

We chose this method so we'd be able to add and remove events without continuous modification to a group of DLLs dispersed geographically among the future installed base, potentially (we hoped) thousands of users. Another benefit was that trivial implementations of the DLL, ones that handled only a subset of the application's events, wouldn't be burdened with the responsibility of providing a list of unused function entry points. The point is that there's a productivity penalty associated with changing a DLL interface once you've designed it. That penalty is proportional to the number of DLLs written to the interface.

Polymorphism and Parameter Packing

A polymorphic interface, although it's admittedly cryptic, limits the cost of evolution while preserving the flexibility of dynamic linking. With some careful planning, the door to the closet of polymorphic anxieties can be kept tightly closed, even at night. Polymorphic parameters, fundamental to Windows itself, simply imply weak typing, the Wirthian anxiety.

To begin, the type of the parameter must be established and agreed on by the sending and receiving parties. The sending party must shoehorn its data into the 32-bit space by means of a compiler cast or bit-shifting sleight of hand. Similarly the receiver, collecting what seems to be an unsigned DWORD value, must prod, nudge, and dissect the value into its respective components.

Notice that the reason for creating strongly typed languages was to relieve programmers of the burden of handling this sort of data typing mentally. Conversely, notice that a great deal of excellent software predates the availability (read: preferred use) of strongly typed language

A Message to Our Subscribers

FROM TIME TO TIME WE MAKE THE BYTE subscriber list available to other companies who wish to send our subscribers material about their products. We take great care to screen these companies, choosing only those who are reputable, and whose products, services or information we feel would be of interest to you. Direct mail is an efficient medium for presenting the latest personal computer goods and services to our subscribers.

Many BYTE subscribers appreciate this controlled use of our mailing list, and look forward to finding information of interest to them in the mail. Used are our subscribers' names and addresses only (no other information we may have is ever given).

While we believe the distribution of this information is of benefit to our subscribers, we firmly respect the wishes of any subscriber who does not want to receive such promotional literature. Should you wish to restrict the use of your name, simply send your request to the following address.

BYTE Magazine
Attn: Subscriber Service
P.O. Box 555
Hightstown NJ 08520

compilers. You can certainly create robust software using polymorphic entry points, but you have to really want to. Failing to properly cast a 16-bit pointer before passing it into a 32-bit integer parameter can result in the compiler incorrectly filling the upper 16 bits of the parameter with a sign extension. Even if the receiving party is able to dereference this new and interesting data type, we can be assured that it does not point to the intended value.

If you're unfortunate enough to be implementing a DLL for which you have no knowledge of the calling application, you'll be able to get some mileage from the new parameter-validation routines that are available in the Windows 3.1 API. Such routines can be used to ensure that a pointer to writable memory is actually so. Alternately, by establishing some clever design standards for parameter passing, additional typing knowledge can be imparted on polymorphic parameters.

Notice how most of the new data structures in Windows 3.1 require that you initialize the first DWORD of the structure with the size of the structure. My guess is that this information makes implementing a thunk-layer (e.g., Windows 3.1 running on Windows NT) that much easier. The thunk can examine the first field of the structure and determine whether a 16- or 32-bit compiler created the structure. I call this approach "better than nothing" or "strong most of the time" typing.

Choosing the other alternative—separate named entry points for each function—solves these problems. Or does it solve them? Essentially, the big problem you face with polymorphic parameters is ensuring that the proper values are packed into parameters (primarily during compilation). If either side of the interface changes without both entities (application and DLL) being aware, problems at run time persist with either solution. If the application adds a new parameter to a function call and the DLL entry point is expecting the old stack frame, the system will die an ugly death.

Creating individual entry points does, of course, make for more readable code. Consider how much more cryptic the Word filters would be with a single entry point. Achieving maximum benefit from multiple entry points is a result of stabilizing the names and/or ordinal identifiers of the exported DLL functions. This allows the specification to be "grown" without affecting earlier incarnations of the DLLs.

Basically, the difference between fewer polymorphic entry points and more numerous typed entry points is the amount of information about the interface that is packaged with the DLL itself. The presence of more information means greater understandability and more opportunities for querying the DLL package (at run time) about its capabilities. Maintaining this extra information means a proliferation of module definition file entries and methods, resulting in a greater "change ripple" when parts of the interface evolve. But perhaps this required effort will shift our emphasis toward getting the interface right the first time, focusing our attention on the value of good software engineering and design. ■

Gen Kiyooka is the founder of Electron Image. He creates Windows software and is the inventor of the RoboHelp Windows help authoring tool and the BugMan debugging agent from Blue Sky Software. You can reach him on BIX c/o "editors" or on CompuServe at 76376,43.

Girl Scouts Care for the Earth

When was the last time you looked at the color of a rainbow or listened to a chorus of spring frogs? Lucky for us millions of girls still do–in the Girl Scouts.

This year, to celebrate our 80th birthday, Girl Scouts nationwide are working to help care for our environment through a national service project called "Girl Scouts Care for the Earth."

Won't you join us? Call your local Girl Scout council to learn how.

GIRL SCOUTS